

Scaling Sparse Matrix Computation for Secure Outsourced Computing

Wujie Xiong¹, Hao Zhou¹, Yutong Ye², Ruoming Jin¹, and Lei Xu¹

¹ Kent State Univeristy

² Beihang Univeristy

Abstract. Sparse General Matrix–Matrix Multiplication (SpGEMM) is a fundamental but computationally intensive operation that underpins many scientific workloads, including numerous AI applications. With the increasing demands for data security, privacy-preserving computation techniques, such as Fully Homomorphic Encryption (FHE), have gained significant attention for their ability to process sensitive data without decryption. Nonetheless, executing SpGEMM within the framework of FHE presents significant challenges. The most effective SpGEMM algorithms exploit matrix sparsity to minimize computational costs; however, FHE obscures both the data values and the sparsity structures. Prior FHE-based privacy-preserving computation frameworks either ignore the inherent sparsity of matrices and rely on dense General Matrix–Matrix Multiplication (GEMM), incurring substantial overhead from redundant homomorphic multiplications, or they attempt to exploit sparsity by encrypting only the non-zero values, which inadvertently exposes sensitive positional information. To address this gap and achieve a better balance between efficiency and privacy, we propose our scheme, an efficient FHE-compatible SpGEMM framework that enables oblivious data and position processing under a Single Instruction Multiple Data (SIMD) scheme. Moreover, we extend our method to support an arbitrary number of sparse matrices (FHE-SpGEMCM). The efficiency analysis shows that our method achieves an average homomorphic computation cost of $(n_A n_B)^2 / n^2 N$, where n_A and n_B represent the number of nonzero elements in A and B respectively, n is the shared inner dimension of the multiplication and N denotes the batch size used in FHE. Experimental results demonstrate that for square matrices of scale 2^9 , our scheme achieves an average speedup of $439.25\times$ and a $10.68\times$ reduction in memory consumption compared to state-of-the-art baselines that ignore sparsity. Furthermore, when the scale increases to 2^{13} , our method yields up to a $1201.77\times$ speedup over baselines that only exploit the sparsity of a single matrix.

1 Introduction

Sparse General Matrix-matrix Multiplication (SpGEMM) [8] is a fundamental mathematical operation that calculates the multiplication of two sparse matrices, and it has various applications that are applied across graph analytics [3, 12, 22],

neural network inference [4, 16, 33], and recommendation systems [20, 27, 38]. By leveraging the fact that most elements of a sparse matrix are zero, SpGEMM algorithms are usually more efficient than general purpose matrix multiplication methods. Classical SpGEMM algorithms include Gustavson’s algorithm [14], outer-product algorithm [6], and many others [8]. Additionally, substantial research has been conducted on employing specialized hardware, such as Graphics Processing Units (GPUs) [23] and Field Programmable Gate Arrays (FPGAs) [25], to accelerate SpGEMM. All these works use information about the positions of nonzero elements to achieve better performance than General Matrix-Matrix Multiplication (GEMM).

Many SpGEMM-based applications process sensitive datasets, such as financial and medical records, precipitating a critical demand for privacy-preserving computation. In these contexts, confidentiality requirements extend beyond the nonzero values to include their locations, as the sparsity pattern itself often reveals proprietary structural information. This scenario is typically modeled as a secure outsourcing problem, where a data owner delegates SpGEMM operations to an untrusted third-party provider. Fully Homomorphic Encryption (FHE) [1] has emerged as a definitive solution for such tasks by enabling computation directly on encrypted data. While adapting FHE for dense matrix multiplication is methodologically straightforward, reconciling comprehensive data protection with the performance benefits of sparsity remains a formidable challenge. Although recent studies have attempted to implement sparse matrix operations under FHE [9, 41], these approaches fail to effectively leverage the joint sparsity of both input matrices to minimize the computational complexity of homomorphic evaluations.

To address this challenge, we propose a flexible and general framework for privacy-preserving SpGEMM under FHE. Our scheme supports sparse matrices with arbitrary shapes and fully exploits sparsity without revealing underlying nonzero patterns. Meanwhile, it is compatible with mainstream FHE schemes, including CKKS [39], BGV [21], and BFV [10], and can seamlessly benefit from future advances in FHE design. The key idea of our method is to pre-align nonzero elements of the input sparse matrices prior to encryption, which preserves the confidentiality of both data values and sparsity structures while enabling efficient sparsity-aware computation. For the accumulation phase, we adopt a $\text{sinc}(\cdot)$ -based polynomial approximation to eliminate data-dependent index matching, thereby enabling full utilization of SIMD packing to amortize computational costs. To the best of our knowledge, our scheme is the first framework that enables efficient outsourcing of SpGEMM under FHE with full sparsity preservation. Theoretically, our framework achieves a worst-case time complexity of $O((n_A n_B)^2 / N)$, while under a random sparsity model, the expected complexity further reduces to $O((n_A n_B)^2 / (n^2 N))$. By using the `Pyfhe1` library, we evaluate our scheme on sparse matrices ranging from 2^4 to 2^{13} . Comprehensive experimental results show that, compared with state-of-the-art FHE-based SpGEMM solutions, our framework delivers up to $439.25\times$ speedup and $10.68\times$ memory re-

duction for square matrices, and achieves up to $1201.77\times$ acceleration on highly non-square matrices.

In summary, the contributions of this paper include:

- We present a FHE-compatible framework for outsourced SpGEMM on matrices with arbitrary dimensions, preserving the confidentiality of both values and sparsity patterns while leveraging sparsity for efficiency.
- We extend the framework to accommodate an arbitrary number of sparse matrices by enabling the structural alignment of matrices directly within the ciphertext. Furthermore, we design a deduplication mechanism to effectively constrain the growth of intermediate product elements.
- We provide a theoretical security analysis under the semi-honest adversary model, validating that our scheme keeps both data confidentiality and structural privacy. Furthermore, we provide rigorous mathematical proofs to verify the tightness of our theoretical upper bound and to establish the average-case complexity.
- We conduct comprehensive evaluations across various matrix scales, demonstrating our scheme consistently outperforms state-of-the-art methods in both computation efficiency and memory consumption.

2 Background and Problem Statement

2.1 Sparse Matrix Multiplication

Sparse General Matrix-Matrix Multiplication (SpGEMM) is a fundamental kernel in linear algebra that computes the product of two matrices containing a significant proportion of zero elements. Unlike dense matrix multiplication, which computes every element pair regardless of value, SpGEMM algorithms aim to reduce computational complexity by performing arithmetic operations only on non-zero entries. Standard optimizations on conventional architectures rely on compressed formats (e.g., CSR, CSC, and COO [11]) to eliminate redundant storage and on dynamic formulations (e.g., row-by-row [8] or outer product [6]) to enhance data reuse.

However, these classical techniques are largely incompatible with Fully Homomorphic Encryption (FHE). First, the data-hiding nature of FHE prohibits data-dependent branching, such as index matching or dynamic memory allocation, which are central to sparse optimizations. Second, FHE relies on ciphertext packing (SIMD) to amortize costs, but the irregular and unstructured access patterns of classical sparse formats conflict with the alignment and rotation operations required in the encrypted domain. Consequently, the high overhead of homomorphic multiplication and the constraints of data-independent execution render conventional SpGEMM algorithms prohibitively expensive for FHE-based privacy-preserving applications.

2.2 Fully Homomorphic Encryption (FHE)

Fully Homomorphic Encryption (FHE) allows computation over encrypted data with encrypted outputs, providing a promising solution for enhancing data privacy and security in public cloud settings. Modern FHE research has established several mainstream schemes tailored for different data types, including BGV and BFV for exact integer arithmetic, TFHE for boolean gate evaluations, and CKKS for approximate fixed-point arithmetic. To achieve functional completeness, these schemes support two fundamental operations.

- Fully Homomorphic Addition (FHE-Add): $\text{Enc}(m_1) + \text{Enc}(m_2) = \text{Enc}(m_1 + m_2)$;
- Fully Homomorphic Multiplication (FHE-Mult): $\text{Enc}(m_1) \times \text{Enc}(m_2) = \text{Enc}(m_1 \times m_2)$.

While FHE-Add is computationally efficient, FHE-Mult is significantly more expensive and introduces substantial noise, necessitating techniques such as re-linearization and bootstrapping to maintain ciphertext correctness. To mitigate these overheads, modern FHE implementations leverage packing (SIMD) techniques, which encode a vector of messages into a single ciphertext. This allows a single homomorphic operation to be applied to multiple plaintext slots simultaneously, effectively amortizing the computational cost and enhancing throughput for data-intensive tasks such as matrix multiplication.

2.3 Security Model and Problem Statement

Participants and assumptions. For our FHE-based outsource sparse matrix multiplication computation, it involves two parties: Client $\mathcal{C}$ and Server $\mathcal{S}$.

- $\mathcal{C}$: The party who owns the input matrices, and is aware of both the values and the coordinates of all nonzero elements. As the data owner and the end user for subsequent operations, $\mathcal{C}$ is assumed to be honest, because any manipulation will undermine its final result.
- $\mathcal{S}$: Processing more powerful computation resources, $\mathcal{S}$ is responsible for executing the encrypted FHE operations. $\mathcal{S}$ is considered as semi-honest, implying it is curious but honest, as it will not generate computation errors intentionally.

High-level protocol. The high-level interaction workflow of $\mathcal{C}$ and $\mathcal{S}$ proceeds as follows: **(i) Element alignment.** $\mathcal{C}$ pre-processes and expands the nonzero elements of input matrices to align them for computation. These elements are then encrypted and transmitted to $\mathcal{S}$. **(ii) Multiplication and accumulation.** $\mathcal{S}$ calculates element-wise multiplication and accumulates the element corresponding to the same indices of the result matrix. **(iii) Decryption and reconstruction.** When receiving the final result from $\mathcal{S}$, $\mathcal{C}$ decrypts and reconstructs the final result matrix.

Security goals. The primary objective is to ensure data confidentiality and access pattern privacy, while fully utilizing sparsity to improve computation efficiency. Specifically, $\mathcal{S}$ should remain oblivious to the values of the nonzero elements and their positions during processing.

3 Detailed Design of Our Scheme

In this section, we present the details of our scheme design in three parts: **(i)** the major tools used in the construction; **(ii)** multiplication of two sparse matrices; and **(iii)** chained sparse matrices multiplication for three or more matrices.

3.1 Major Building Blocks

Matrices Storage The storage format of sparse matrices can significantly affect the efficiency of the multiplication operation. Our framework uses both compressed sparse column (CSC) and compressed sparse row (CSR) formats [11]. To facilitate the description, we consider the multiplication of two matrices $A \times B$.

Sparse matrix $A \in \mathbb{R}^{m \times n}$ contains n_A nonzero elements and is stored using the CSC format, where $\mathbb{R}$ denotes the domain of real numbers. Specifically, A is represented by three arrays³ $A_{\text{val}} \in \mathbb{R}^{n_A}$, $A_{\text{row}} \in \{1, \dots, m\}^{n_A}$, and $A_{\text{colptr}} \in \{p \in \mathbb{Z}^{n+1} \mid 1 = p_1 \leq p_2 \leq \dots \leq p_{n+1} = n + 1\}$. These arrays satisfy:

$$\begin{cases} A_{\text{val}}[k] = A_{i,j} & \text{if the } k\text{-th stored element corresponds to } A_{i,j}, \\ A_{\text{row}}[k] = i & \text{if the } k\text{-th stored element lies in row } i. \end{cases}$$

For each column j , A 's nonzero elements are stored in the contiguous segment of $A_{\text{val}}[k]$, where $A_{\text{colptr}}[j] \leq k < A_{\text{colptr}}[j + 1]$.

Similarly, the sparse matrix $B \in \mathbb{R}^{n \times s}$ with n_B nonzero elements is stored using the CSR format and represented using three arrays: $B_{\text{val}} \in \mathbb{R}^{n_B}$, $B_{\text{col}} \in \{1, \dots, s\}^{n_B}$, and $B_{\text{rowptr}} \in \{p' \in \mathbb{Z}^{n+1} \mid 1 = p'_1 \leq p'_2 \leq \dots \leq p'_{n+1} = n + 1\}$. These arrays satisfy:

$$\begin{cases} B_{\text{val}}[k] = B_{i,j} & \text{if the } k\text{-th stored element corresponds to } B_{i,j}, \\ B_{\text{col}}[k] = j & \text{if the } k\text{-th stored element lies in column } j. \end{cases}$$

For each row i , the nonzero elements of B are stored in the contiguous segment of $B_{\text{val}}[k]$, where $B_{\text{rowptr}}[i] \leq k < B_{\text{rowptr}}[i + 1]$.

Equality Check Comparison is a critical operation for our scheme, i.e., given two ciphertexts $\text{Enc}(x)$ and $\text{Enc}(y)$, we need to decide whether the corresponding plaintexts x and y are equal. Instead of using costly scheme-switching approach [26], our scheme adopts the $\text{sinc}(\cdot)$ function for equality check [34], which is defined as:

$$\text{sinc}(m) = \frac{\sin(m\pi)}{(m\pi)} = \prod_{n=1}^{\infty} \left(1 - \frac{m^2}{n^2}\right).$$

³ For simplicity, indices and pointers of matrix start from 1.

This function has three essential properties: **(i)** $\text{sinc}(0) = 1$; **(ii)** $\text{sinc}(m) = \text{sinc}(-m)$; and **(iii)** $\text{sinc}(m) = 0$ for every $m \in \mathbb{Z} \setminus \{0\}$. Unlike the $\text{sgn}(\cdot)$ function [17], $\text{sinc}(\cdot)$ is a continuous function with no discontinuities and low multiplicative depth, which means it will not suffer from severe Gibbs phenomena [13] using polynomial approximation. Given its ideal characteristics, the $\text{sinc}(\cdot)$ function could serve as an effective zero equality test under the CKKS scheme.

To efficiently evaluate the $\text{sinc}(\cdot)$ function under the CKKS scheme, we approximate it using Chebyshev polynomials [35]. The target function is first linearly rescaled to the interval $[-1, 1]$ and the polynomials are defined by the recurrence relations $T_0(t) = 1$, $T_1(t) = t$, $T_{2n}(t) = 2T_n(t)^2 - 1$, and $T_{2n+1}(t) = 2T_n(t)T_{n+1}(t) - t$, which satisfy the identity $T_k(\cos \theta) = \cos(k\theta)$.

The approximation is then constructed by interpolating the rescaled function at the Chebyshev nodes:

$$t_i = \cos\left(\frac{(i + \frac{1}{2})\pi}{n}\right), \quad i = 0, \dots, n-1, \quad (1)$$

yielding a truncated Chebyshev expansion:

$$p_{n-1}(t) = \sum_{k=0}^{n-1} c_k T_k(t), \quad (2)$$

where the coefficients c_k are efficiently obtained via a discrete cosine transform. The resulting polynomial is then mapped back to the original variable to obtain a polynomial approximation $P(x) = p_{n-1}\left(\frac{2x-(a+b)}{b-a}\right)$ of $\text{sinc}(x)$ over $[a, b]$. Moreover, to maintain low multiplicative depth while efficiently evaluating $P(x)$, we employ a modified Paterson-Stockmeyer scheme [32] to reduce computation cost and numerical bias.

FHE Vector Extension In many FHE applications, particularly those involving cross-slot computations, it is often necessary to replicate a small encrypted vector across all slots of a ciphertext. Under the FHE scheme, we achieve this efficiently by leveraging the SIMD structure of ciphertexts and implementing a Homomorphic Vector Extension algorithm (Algorithm 1) based on cyclic rotations and additions.

Compared to a naive linear extension, this approach offers two significant advantages: **(i)** Logarithmic Complexity: It requires only $\lceil \log_2(L_2/L_1) \rceil$ homomorphic rotations and additions, which is critical since rotation is a computationally expensive operation in FHE; **(ii)** Noise Management: By minimizing the number of operations, it preserves the ciphertext's budget for subsequent functional evaluations, such as the $\text{sinc}(\cdot)$ function described in 3.1.

3.2 FHE-SpGEMM: Case of Multiplication of Two Matrices

We first consider the scenario where the user has two sparse matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times s}$, stored in plaintext using CSC and CSR formats respectively, and

Algorithm 1 Homomorphic Vector Extension

```
1: procedure VECTOREXTENSION( $ct, L_1, L_2$ )
2:                                      $\triangleright$   $ct$ : Ciphertext,  $L_1$ : initial length,  $L_2$ : final length
3:    $ct_{ext} \leftarrow ct$                                       $\triangleright$  Initialize the result
4:    $step \leftarrow L_1$ 
5:   while  $2 \times step \leq L_2$  do
6:      $ct_{rot} \leftarrow \text{Rotate}(ct_{ext}, -step)$               $\triangleright$  Left cyclic shift
7:      $ct_{ext} \leftarrow \text{Add}(ct_{ext}, ct_{rot})$ 
8:      $step \leftarrow 2 \times step$ 
9:   end while
10:  if  $step < L_2$  then
11:     $ct_{rot} \leftarrow \text{Rotate}(ct_{ext}, -(L_2 - step))$ 
12:     $ct_{ext} \leftarrow \text{Add}(ct_{ext}, ct_{rot})$ 
13:  end if
14:  return  $ct_{ext}$ 
15: end procedure
```

wants to utilize the service provider to carry out the multiplication $A \times B$. Unlike the conventional approach of storing A in CSR and B in CSC, which offers no guarantee that the aligned elements in their respective compressed structures share a matching inner dimension for valid multiplication, the core intuition relies on a pre-alignment strategy that ensures every paired non-zero element is eligible for an immediate multiplicative operation. This alignment allows us to fully harvest inherent sparsity to minimize both homomorphic multiplications and costly rotations. While this introduces a subsequent accumulation phase, we strategically delegate this computationally heavy task to the server $\mathcal{S}$, whose superior computation power makes the overhead of outsourced method a highly acceptable trade-off.

Element Alignment Before sending the encrypted matrices to $\mathcal{S}$ for multiplication, $\mathcal{C}$ first expands the matrices A and B to enable efficient matching in ciphertext form.

Expansion length. For the matrix product:

$$C_{ij} = \sum_{k=1}^n A_{i,k} B_{k,j}, \quad (3)$$

a nonzero product arises only from pairs of candidates that share the same inner index k . Thus for each inner index k , we just consider **column** k of A and **row** k of B .

For column k of A , let $k_A = A_{\text{colptr}}[k+1] - A_{\text{colptr}}[k]$ be the number of nonzero elements in that column, and let $p_A = A_{\text{colptr}}[k]$ be the starting position of that column.

For row k of B , let $k_B = B_{\text{rowptr}}[k+1] - B_{\text{rowptr}}[k]$ be the number of nonzero elements in that row, and let $p_B = B_{\text{rowptr}}[k]$ be the starting position of that row.

The expansion length for index k is then given by $\ell_k = k_A k_B$, which represents the total number of candidate pairs produced by pairing each nonzero element in column k of A with each nonzero in row k of B .

Expanded index mapping. For each position $\ell = 1, \dots, \ell_k$, define:

$$r_k(\ell) = (\ell - 1) \bmod k_A, \quad q_k(\ell) = \left\lfloor \frac{\ell - 1}{k_A} \right\rfloor.$$

Using these offsets, the expanded indices that map back to the original CSC and CSR structures are:

$$k_{A,k}^{(\ell)} = p_A + r_k(\ell), \quad k_{B,k}^{(\ell)} = p_B + q_k(\ell).$$

Thus the expanded sequences for each inner index k given in Eq. (3) are:

$$\begin{aligned} A'_{\text{val}}^{(k)}[\ell] &= A_{\text{val}}[k_{A,k}^{(\ell)}], & A'_{\text{row}}^{(k)}[\ell] &= A_{\text{row}}[k_{A,k}^{(\ell)}], \\ B'_{\text{val}}^{(k)}[\ell] &= B_{\text{val}}[k_{B,k}^{(\ell)}], & B'_{\text{col}}^{(k)}[\ell] &= B_{\text{col}}[k_{B,k}^{(\ell)}]. \end{aligned}$$

In other words, for the inner index k , the alignment of A_{val} yields the following results (Eq. (4)), where a sub-sequence includes k_A elements and the same sub-sequence is repeated k_B times.

$$\begin{aligned} A'_{\text{val}} \left[\sum_{t=1}^{k-1} \ell_t, \sum_{t=1}^{k-1} \ell_t + 1, \dots, \sum_{t=1}^{k-1} \ell_t + \ell_k - 1 \right] &= \\ \underbrace{A_{\text{val}}[p_A + 0], \dots, A_{\text{val}}[p_A + k_A - 1]}_{k_A \text{ elements}}, & (4) \\ \underbrace{A_{\text{val}}[p_A + 0], \dots, A_{\text{val}}[p_A + k_A - 1]}_{k_A \text{ elements}}. \end{aligned}$$

Moreover, the aligned B_{val} is defined as follows (Eq. (5)), where each sub-sequence includes k_A elements of the same value and there are k_B different sub-sequences. Following a similar procedure, we construct A'_{row} and B'_{col} , with the specific workflow is illustrated in Figure ??.

$$\begin{aligned} B'_{\text{val}} \left[\sum_{t=1}^{k-1} \ell_t, \sum_{t=1}^{k-1} \ell_t + 1, \dots, \sum_{t=1}^{k-1} \ell_t + \ell_k - 1 \right] &= \\ \underbrace{B_{\text{val}}[p_B + 0], \dots, B_{\text{val}}[p_B + 0]}_{k_A \text{ elements}}, & (5) \\ \underbrace{B_{\text{val}}[p_B + k_B - 1], \dots, B_{\text{val}}[p_B + k_B - 1]}_{k_A \text{ elements}}. \end{aligned}$$

Element-wise Products and Indexing After the matching step, the expanded vectors are encrypted and transmitted to $\mathcal{S}$, which performs element-wise multiplications. For convenience, we assume each vector obtained contains ℓ elements.

Given the expanded value vectors $\text{Enc}(A'_{\text{val}})$ and $\text{Enc}(B'_{\text{val}})$, $\mathcal{S}$ computes the Hadamard product⁴, where $\text{val} = A'_{\text{val}} \odot B'_{\text{val}}$:

$$\text{Enc}(\text{val}) = \text{Enc}(A'_{\text{val}}) \odot \text{Enc}(B'_{\text{val}}), \quad (6)$$

We assign a linearized output $\text{Enc}(\text{ind})$ to each pair corresponding to the coordinate c , where the index ranges from 1 to $m \times s$:

$$\text{Enc}(\text{ind}[c]) = s(\text{Enc}(A'_{\text{row}}[c]) - 1) + \text{Enc}(B'_{\text{col}}[c]). \quad (7)$$

Indicator matrices from index vector. For simplicity, let $\text{Enc}(\text{ind}) = \text{Enc}(\mathbf{a}) = [a_1, a_2, \dots, a_\ell]$. We follow the replication-and-transposition framework developed in [30] to homomorphically transform $\text{Enc}(\mathbf{a})$ into two index matrices $\text{Enc}(\text{ind}_1)$ and $\text{Enc}(\text{ind}_2)$. Specifically, $\text{Enc}(\text{ind}_1)$ is generated by applying recursive row replications (ReplR) to $\text{Enc}(\mathbf{a})$, whereas $\text{Enc}(\text{ind}_2)$ is obtained by first performing a row-to-column transposition via recursive homomorphic rotations (TransR), and subsequently applying recursive column replications (ReplC).

$$\begin{aligned} \text{Enc}(\text{ind}_1) &= \begin{pmatrix} a_1 & a_2 & \cdots & a_\ell \\ a_1 & a_2 & \cdots & a_\ell \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \cdots & a_\ell \end{pmatrix}, \\ \text{Enc}(\text{ind}_2) &= \begin{pmatrix} a_1 & a_1 & \cdots & a_1 \\ a_2 & a_2 & \cdots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_\ell & a_\ell & \cdots & a_\ell \end{pmatrix}. \end{aligned} \quad (8)$$

To compare indices in an encrypted form, we leverage the $\text{sinc}(\cdot)$ equality check function described in Section 3.1. If $a_i = a_j$, $\text{sinc}(a_i - a_j) = 1$, otherwise $\text{sinc}(a_i - a_j) = 0$. Thus, we can obtain the encrypted equality matrix EM by leveraging the SIMD scheme. It should be noted that if $\text{sinc}(\cdot)$ takes a vector as its argument throughout this paper, we denote it a SIMD operation where the $\text{sinc}(\cdot)$ function is applied to each element of the vector individually.

$$\begin{aligned} \text{Enc}(\text{EM}) &= \text{sinc}(\text{Enc}(\text{ind}_1 \ominus \text{ind}_2)) = \\ &\begin{pmatrix} \text{sinc}(a_1 - a_1) & \text{sinc}(a_1 - a_2) & \cdots & \text{sinc}(a_1 - a_\ell) \\ \text{sinc}(a_2 - a_1) & \text{sinc}(a_2 - a_2) & \cdots & \text{sinc}(a_2 - a_\ell) \\ \vdots & \vdots & \ddots & \vdots \\ \text{sinc}(a_\ell - a_1) & \text{sinc}(a_\ell - a_2) & \cdots & \text{sinc}(a_\ell - a_\ell) \end{pmatrix} \end{aligned} \quad (9)$$

⁴ $\odot$ denotes the Hadamard product under FHE.

The matrix $\text{Enc}(\text{EM})$ is symmetric and it satisfies $\text{EM}_{ii} = 1$ for all i . Its entries indicate that encrypted indices are equal; thus, EM functions could be used as an encrypted equality-check operator. Moreover, to avoid the quadratic memory overhead $O(\ell^2)$ in practice, the matrix $\text{Enc}(\text{EM})$ is treated as a logical construct. During the accumulation phase, $\mathcal{S}$ computes each wrap-around diagonal vector M_k via a sliding window. This ensures that the peak memory consumption remains $O(\ell \cdot N)$, where ℓ is the expanded length and N is the number of SIMD slots.

Diagonal extraction of the equality matrix. Given the equality matrix $\text{Enc}(\text{EM}) = (a_{ij})_{1 \leq i, j \leq \ell}$, we reorganize its diagonals to adapt to diagonal-rotation multiplication.

Wrap-around diagonal vectors. For any integer offset $k \in \mathbb{Z}$, we define the wrap-around diagonal:

$$M_k = (a_{i, (i+k-1 \bmod \ell)+1})_{i=1}^{\ell}.$$

This vector collects the elements on the $(i, i+k)$ diagonal of EM, with indices evaluated modulo ℓ so that out-of-range positions wrap around. Here, we denote positive offsets $k > 0$ produce right-shifted diagonals, and negative offsets $k < 0$ produce left-shifted diagonals.

Modulo reduction of negative offsets. Since the column index is computed through modulo- ℓ arithmetic, negative offsets do not produce new diagonals. Indeed, for $k > 0$,

$$(i - k - 1) \bmod \ell = (i + (\ell - k) - 1) \bmod \ell,$$

which implies that the diagonal obtained from offset $-k$ is identical to that obtained from offset $\ell - k$. Thus,

$$M_{-k} = M_{\ell-k}, \quad k = 1, \dots, \ell - 1.$$

Consequently, although offsets from $-(\ell - 1)$ to $(\ell - 1)$ are allowed conceptually, only the nonnegative offsets $M_0, M_1, \dots, M_{\ell-1}$ correspond to distinct wrap-around diagonals.

Redundancy from symmetry. Due to the symmetry of EM , the diagonals M_k and $M_{\ell-k}$ are identical up to cyclic rotation. More precisely,

$$M_{\ell-k} = M_k = \text{Rot}(M, k), \quad k = 1, \dots, \ell - 1.$$

Therefore, each pair $(M_k, M_{\ell-k})$ carries the same information modulo rotation, and only half of the diagonals are unique. The set of unique diagonals is :

$$M_0, M_1, \dots, M_{\lfloor (\ell-1)/2 \rfloor}.$$

Diagonal collection. Conceptually, one may list all wrap-around diagonals in symmetric order,

$$(M_{-(\ell-1)}, \dots, M_{-1}, M_0, M_1, \dots, M_{\ell-1}),$$

but modulo reduction and symmetry imply that all negative-offset diagonals are either identical to or cyclic rotations of the positive-offset diagonals. Thus, we store only:

$$M = (M_0, M_1, \dots, M_{\ell-1}),$$

and for computation, we use only the unique subset:

$$(M_0, M_1, \dots, M_{\lfloor (\ell-1)/2 \rfloor}).$$

Accumulation Since the diagonal M_0 consists of identity elements (all ones), the accumulation can be performed homomorphically by aggregating the unique diagonals:⁵

$$\begin{aligned} \text{Enc}(\text{result}) = & \\ \text{Enc}(\text{val}) + \sum_{k=1}^{\lfloor (\ell-1)/2 \rfloor} & (M_k \odot \text{Rot}(\text{val}, -k) + \text{Rot}(M_k \odot \text{val}, k)) \\ + \begin{cases} 0, & \ell \text{ odd,} \\ M_{\ell/2} \odot \text{Rot}(\text{val}, \ell/2), & \ell \text{ even.} \end{cases} & \end{aligned} \quad (10)$$

The proposed accumulation pattern in *Eq. (10)* yields several critical computational advantages while keep data confidential: **(i)** Noise Management. Since terms share a synchronized multiplicative depth, a single rescaling suffices to maintain precision. **(ii)** SIMD Efficiency. This structure allows $\mathcal{S}$ to overlap rotation latencies, enabling concurrent operations.

Removing duplicates. While the most efficient way to handle redundancies in the $\text{Enc}(\text{result})$ vector is to let $\mathcal{C}$ perform de-duplication using ind data, this approach fails when considering sparse matrix chain multiplication. To address this, we present an FHE-based de-duplication scheme for the $\text{Enc}(\text{result})$ vector to enable following ciphertext processing.

For $\text{Enc}(\text{EM})$, here's a SIMD-friendly occurrence accumulation method, we can use it to determine the number of occurrences of each element up to its current position in the vector $\text{Enc}(\text{result})$, denote the result of Algorithm 2 as $\text{Enc}(\mathbf{r})$. It is important to emphasize that the masks are predefined plaintext vectors, thus the masking operation is significantly more efficient than standard homomorphic multiplications. This allows $\mathcal{S}$ to process the occurrence sums with negligible computational overhead.

Based on the properties of vector $\text{Enc}(\mathbf{r})$, it can be observed that if $r[i] \neq 0$, there must exist another element with an identical value as $\text{val}[i]$. Consequently, a deduplication mask $\text{Enc}(\mathbf{m})$ is produced by evaluating each element in vector $\mathbf{r}$ against the value 0.

$$\text{Enc}(\mathbf{m}) = \text{sinc}(\text{Enc}(\mathbf{r})) \quad (11)$$

⁵ For $\text{Rot}(\mathbf{v}, k)$, $k > 0$ means right shift, and $k < 0$ means left shift.

Algorithm 2 Occurrence Accumulation

```
1: procedure GETOCCSUMS(EncRows(A))    ▷  $ct_i$  packs row  $i$ :  $(a_{i,1}, \dots, a_{i,n})$  in  
   SIMD slots  
2:    $ct_{\text{sum}} \leftarrow \text{Enc}(0, \dots, 0)$   
3:   for  $i \leftarrow 1$  to  $n$  do  
4:      $mask_i \leftarrow [0, \dots, 0, \underbrace{1, \dots, 1}_{n-i}]$   
5:                               ▷ Predefined plaintext mask, independent of input values  
6:      $ct \leftarrow \text{MulPlain}(ct_i, mask_i)$     ▷ Mask out lower triangle and diagonal.  
7:      $ct_{\text{sum}} \leftarrow \text{Add}(ct_{\text{sum}}, ct)$   
8:   end for  
9:   return  $ct_{\text{sum}}$     ▷ Column sums of the upper matrix  
10: end procedure
```

Thus, we obtain the final de-duplicated vectors $result'$ and ind' , which can be sent directly to the user side $\mathcal{C}$ for decryption.

$$\begin{aligned} \text{Enc}(result') &= \text{Enc}(result) \odot \text{Enc}(m), \\ \text{Enc}(ind') &= \text{Enc}(ind) \odot \text{Enc}(m), \\ \text{Enc}(A'_{\text{row}}) &= \text{Enc}(A_{\text{row}}) \odot \text{Enc}(m), \\ \text{Enc}(B'_{\text{col}}) &= \text{Enc}(B_{\text{col}}) \odot \text{Enc}(m). \end{aligned} \tag{12}$$

3.3 FHE-SpGEMCM: Case of Chained Matrices Multiplication

In this subsection, we describe the approach for computing the product of an arbitrary number of sparse matrices, which is built upon the two matrices multiplication method FHE-SpGEMM.

Without loss of generality, we consider the FHE matrix multiplication of ABC , where $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times s}$ and $C \in \mathbb{R}^{s \times t}$. If the multiplication of three matrices can be computed successfully, it follows by induction that it can be extended to an arbitrary chain of matrices. In the multiplication of the matrix chain ABC , to fully leverage the algorithm in Section 3.2, we consider the product AB as an intermediate result to be multiplied by C . Therefore, the problem is simplified as given an encrypted matrix $X = AB \in \mathbb{R}^{m \times s}$, how to perform multiplication with a subsequent matrix $C \in \mathbb{R}^{s \times t}$ with n_C nonzero elements.

It should be noted that, while the initial matrices can be perfectly pre-aligned by the user $\mathcal{C}$ to maximize slot utilization, the intermediate product X is encrypted on the server $\mathcal{S}$, and C remains as plaintext on $\mathcal{C}$ before sent to $\mathcal{S}$. Consequently, to preserve data confidentiality and security, pre-alignment is no longer fully feasible, which inevitably sacrifices some computational efficiency.

For the intermediate product $X = AB \in \mathbb{R}^{m \times s}$, the computation yields deduplicated vectors $\text{Enc}(result')$ and $\text{Enc}(ind')$. By applying the deduplication mask $\text{Enc}(m)$, our scheme enforces a homomorphic sparsity restoration, logically zeroing out redundant entries to ensure only unique intermediate products remain active. This logical reset effectively optimizes the input density for subsequent

operations. Furthermore, $\text{Enc}(X_{\text{row}}) = \text{Enc}(A'_{\text{row}})$ and $\text{Enc}(X_{\text{col}}) = \text{Enc}(B'_{\text{col}})$, and there are one-to-one correspondences among them at each vector's index. For $C \in \mathbb{R}^{s \times t}$, here we still adopt CSR format for the storage on $\mathcal{C}$, getting three vectors C_{val} , C_{col} and C_{rowptr} :

$$\begin{aligned} C_{\text{val}} &\in \mathbb{R}^{n_C}, & C_{\text{col}} &\in \{1, \dots, t\}^{n_C}, \\ C_{\text{rowptr}} &\in \{p' \in \mathbb{Z}^{s+1} \mid 1 = p'_1 \leq p'_2 \leq \dots \leq p'_{s+1} = s+1\}. \end{aligned} \quad (13)$$

Before transmission to $\mathcal{S}$, since C resides on $\mathcal{C}$, it possesses the alignment length $\ell_X = \sum_{k=1}^n \ell_k$, each element in vector C is replicated ℓ_X times consecutively.

$$\begin{aligned} C'_{\text{val}} &= \overbrace{[C_{\text{val}}[0], \dots, C_{\text{val}}[0]]}^{\ell_X \text{ elements}}, \dots, \overbrace{[C_{\text{val}}[n_C - 1], \dots, C_{\text{val}}[n_C - 1]]}^{\ell_X \text{ elements}}, \\ C'_{\text{row}} &= \overbrace{[C_{\text{row}}[0], \dots, C_{\text{row}}[0]]}^{\ell_X \text{ elements}}, \dots, \overbrace{[C_{\text{row}}[n_C - 1], \dots, C_{\text{row}}[n_C - 1]]}^{\ell_X \text{ elements}}, \\ C'_{\text{col}} &= \overbrace{[C_{\text{col}}[0], \dots, C_{\text{col}}[0]]}^{\ell_X \text{ elements}}, \dots, \overbrace{[C_{\text{col}}[n_C - 1], \dots, C_{\text{col}}[n_C - 1]]}^{\ell_X \text{ elements}}. \end{aligned} \quad (14)$$

To propagate the computation to matrix $C \in \mathbb{R}^{s \times t}$, we utilize the restored vectors along with their corresponding indices $\text{Enc}(X_{\text{row}})$ and $\text{Enc}(X_{\text{col}})$ for X . $\mathcal{S}$ then expands these intermediate ciphertexts using Algorithm 1 to a total length of $\ell_X \cdot n_C$. This ensures that the active elements are correctly replicated to align with the plaintext entries of C provided by $\mathcal{C}$.

$$\begin{aligned} \text{Enc}(\text{result}_X) &= \text{VectorExtension}(\text{Enc}(\text{result}'), \ell_X, \ell_X \cdot n_C), \\ \text{Enc}(X_{\text{row}}) &= \text{VectorExtension}(\text{Enc}(X_{\text{row}}), \ell_X, \ell_X \cdot n_C), \\ \text{Enc}(X_{\text{col}}) &= \text{VectorExtension}(\text{Enc}), \end{aligned}$$

Following this processing (Figure ??), the data structures are aligned with the requirements of FHE-SpGEMM. This consistency allows us to leverage the methods (as described in Section 3.2 and Section 3.2) using the improved formula below to obtain the final result, where the index ranges from 1 to $m \times t$:

$$\begin{aligned} \text{Enc}(\text{val}) &= \text{Enc}(\text{result}_X) \odot \text{Enc}(C'_{\text{val}}) \odot \text{sinc}(\text{Enc}(X_{\text{col}}) - \text{Enc}(C'_{\text{row}})) \\ \text{Enc}(\text{ind}[c]) &= t(\text{Enc}(X_{\text{row}}[c]) - 1) + \text{Enc}(C'_{\text{col}}[c]). \end{aligned} \quad (15)$$

3.4 Packing Strategies

To achieve high computational throughput and minimize memory consumption, both the element-wise multiplication (Section 3.2) and the accumulation stage (Section 3.2) are implemented using the SIMD capabilities of the underlying FHE scheme.

Packing for FHE-SpGEMM. For the initial multiplication $A \times B$, our scheme utilizes a static encoding strategy where the client $\mathcal{C}$ partitions the pre-aligned

value vectors $(A_{\text{val}}, B_{\text{val}})$ and coordinate vectors $(A_{\text{row}}, B_{\text{col}})$ into $\lceil \ell/N \rceil$ ciphertexts, where N is typically 2^{13} to 2^{15} .

On the server side $\mathcal{S}$, the indicator matrices $\text{ind}_1, \text{ind}_2$ and the equality matrix EM are never explicitly materialized as 2D structures. Instead, they are treated as conceptual objects implicitly represented by N -length diagonal vectors M_k . These diagonals are extracted via homomorphic rotations within the SIMD slots, allowing the accumulation stage (Eq.9) to update N elements concurrently. This strategy transforms the logical $O(\ell^2)$ complexity of indexing into a hardware-friendly $O(\ell^2/N)$ operations, maintaining high slot utilization throughout the two-matrix multiplication. Crucially, the SIMD-friendly accumulation (Eq.10) enables the lazy relinearization strategy detailed in Section 3.2, ensuring high slot utilization while minimizing the homomorphic maintenance overhead.

Packing for FHE-SpGEMCM. In the chained case of multiplying three matrices ABC , the strategy shifts to dynamic server-side alignment to handle encrypted intermediate results. After the de-duplication of AB , our scheme applies the mask $\text{Enc}(\mathbf{m})$ to perform a logical reset on $\text{Enc}(\text{result}')$ and $\text{Enc}(\text{ind}')$. This operation zero-out redundant slots and effectively restores the vector’s sparsity in its packed state without requiring decryption. Note that this does not physically compress the ciphertext. The redundant entries remain in the fixed-length slot structure to avoid potential side-channel leakage. From the server’s perspective, the ciphertext remains a uniform encrypted block, without revealing which indices were masked. Furthermore, to propagate the computation to matrix C , the server $\mathcal{S}$ executes a homomorphic VectorExtension (Algorithm 1) using a recursive sequence of rotations and additions to replicate intermediate entries $\ell_X \cdot n_C$ times. This evaluation significantly reduces the multiplicative depth and noise growth, ensuring the efficiency of the matrix chain processing.

4 Analysis and Evaluation

In this section, we first analyze the security features of our scheme and then provide both theoretical and experimental evaluations of the performance of our scheme.

4.1 Security Analysis and Privacy Guarantees

Threat model. For our scheme, we consider a **semi-honest** (honest-but-curious) adversary model, which is a standard assumption in outsourced computation [7, 15, 18]. In this setting, $\mathcal{S}$ correctly executes the assigned FHE-based homomorphic calculation tasks and does not deviate from the protocol. However, the server $\mathcal{S}$ is curious about the data that it may attempt to infer private information about the underlying sparse matrix from its view, which consists of two value vectors $\text{Enc}(A'_{\text{val}}), \text{Enc}(B'_{\text{val}})$ and two index vectors $\text{Enc}(A'_{\text{row}}), \text{Enc}(B'_{\text{col}})$.

We assume that the adversary does not have access to the secret key SK and has no out-of-band information that could compromise the semantic security of the FHE scheme.

Security analysis and privacy guarantees. We analyze the security and privacy guarantees of our proposed scheme under the semi-honest threat model. Our analysis demonstrates that the server learns no information about the underlying sparse matrices beyond the public parameter ℓ and the dimensions of the matrices.

Data confidentiality. The core confidentiality of our scheme is based on the semantic security (IND-CPA) of the underlying FHE scheme. All four vectors transmitted to $\mathcal{S}$ are encrypted using $\mathcal{C}$'s public key. Based on the Ring Learning with Errors (RLWE) hardness assumption, these ciphertexts are computationally indistinguishable from random noise to any adversary without the secret key SK . Therefore, $\mathcal{S}$ cannot retrieve the plaintext values of the nonzero elements during storage or computation.

Structural privacy. our scheme achieves robust structural privacy by breaking the deterministic correlation between the observable execution time and the underlying matrix sparsity. The expansion length $\ell = \sum_{k=1}^n k_A k_B$ functions as an inherent structural obfuscation layer. Since ℓ is a joint product of the alignment interactions between A and B , it induces a many-to-one mapping: a single observed ℓ is consistent with exponentially many possible matrix supports. The server cannot uniquely decompose this aggregate sum to reconstruct the specific row or column indices or the precise distribution of nonzero elements. Thus, the framework achieves obliviousness: the mask ℓ conceals n_A and n_B , preventing the untrusted server $\mathcal{S}$ from reconstructing their structure.

Data-oblivious equality check. To prevent privacy leakage via memory side-channel, our method ensures a data-oblivious scheme. Traditional sparse computation relies on conditional branching (e.g., if $(a == b)$), which triggers data-dependent instruction flows. In our FHE-based construction, we transform these selective operations into an oblivious arithmetic circuit. We implement the equality test using a $\text{sinc}(\cdot)$ function (Section 3.1) via Chebyshev polynomial approximation. Since polynomial evaluation in the FHE domain consists of a deterministic sequence of homomorphic additions, multiplications, and rotations, the computational cost is fixed. Regardless of the underlying plaintext values, memory access patterns of $\mathcal{S}$ remain identical, ensuring deterministic and data-independent execution patterns at the protocol level.

4.2 Theoretical Complexity Analysis

Here we analyze the computation complexity of our scheme.

Lemma 1. *Given two sparse matrices $A \in \mathbb{R}^{m \times n}$ with n_A nonzero elements and $B \in \mathbb{R}^{n \times s}$ with n_B nonzero elements. The encrypted element-wise multiplication has at most $n_A n_B$ elements.*

Proof. First, define $\ell = \sum_{k=1}^n k_A k_B$, where k_A means A has k_A nonzero elements for column k and k_B means B has k_B nonzero elements for row k . Then we could have $n_A = \sum_{k=1}^n k_A$ and $n_B = \sum_{k=1}^n k_B$.

Therefore, $\ell = \sum_{k=1}^n k_A k_B \leq \sum_{k=1}^n n_A k_B = n_A \sum_{k=1}^n k_B = n_A n_B$.

Consequently, $\ell' = \ell * n_C \leq n_A n_B n_C$ for the FHE-SpGEMCM method. Note that when ℓ reaches its upper bound, this corresponds to the extreme case where all n_A nonzero elements of matrix A lie in a single column and all n_B nonzero elements of matrix B lie in a single row.

Theorem 1. *Under the standard random sparsity model, where the supports of A and B are drawn uniformly at random with independent placements, the expected expansion length satisfies $\mathbb{E}[\ell] = \frac{n_A n_B}{n}$.*

Proof. Under uniform random placement, each column of A contains $\mathbb{E}[k_A] = n_A/n$ nonzero elements in expectation, and each row of B contains $\mathbb{E}[k_B] = n_B/n$.

By independence, $\mathbb{E}[k_A k_B] = \mathbb{E}[k_A] \mathbb{E}[k_B] = \frac{n_A}{n} \cdot \frac{n_B}{n}$. Summing over $k = 1, \dots, n$ gives $\mathbb{E}[\ell] = \sum_{k=1}^n \mathbb{E}[k_A k_B] = n \cdot \frac{n_A n_B}{n^2} = \frac{n_A n_B}{n}$.

Theorem 2. *Given a standard random sparsity model, let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times s}$ where the nonzero positions of each matrix are chosen uniformly at random, the event that ℓ reaches its upper bound occurs with probability $P \approx n^{2-(n_A+n_B)}$.*

Proof. Out of $\binom{mn}{n_A}$ possible supports, $n \binom{m}{n_A}$ lie entirely within one column, yielding $P_A = \frac{n \binom{m}{n_A}}{\binom{mn}{n_A}}$. Similarly, $P_B = \frac{n \binom{s}{n_B}}{\binom{ns}{n_B}}$. Because P_A and P_B are independent, we have $P = P_A P_B$. Using $\binom{mn}{n_A} \approx (mn)^{n_A}/n_A!$ and $\binom{m}{n_A} \approx m^{n_A}/n_A!$, it gives $P_A \approx n^{1-n_A}$ and hence $P \approx n^{2-(n_A+n_B)}$.

Theorem 3. *Given an encrypted vector v of length ℓ , our scheme requires $O(\ell^2/N)$ FHE operations where N denotes the batch size of FHE scheme.*

Proof. The computational efficiency of Sections 3.2 and 3.2 are analyzed primarily through the lens of homomorphic operation overhead, as ciphertext rotations (Rot) and multiplications (Mul) constitute the primary performance bottlenecks in FHE.

In each diagonal of the EM matrix, the algorithm executes two critical rotation operations: one to generate the shifted index vector $\text{Rot}(\text{Enc}(\text{val}), k)$ and another to align the accumulation terms via $\text{Rot}(\text{Mul}(\text{Enc}(M_k), \text{Enc}(\text{val})), -k)$. Furthermore, each iteration involves two explicit homomorphic multiplications for accumulation. To evaluate the $\text{sinc}(\cdot)$ function used for equality checking, we leverage the Paterson–Stockmeyer (P-S) optimization [34]. Given a polynomial of degree d used to approximate $\text{sinc}(\cdot)$ operator, the P-S method reduces the number of non-scalar homomorphic multiplications to $O(\sqrt{d})$, significantly lowering the multiplicative depth compared to the standard $O(d)$ approach.

From a global complexity perspective, let ℓ denote the total length of the index vector and N denote the number of available SIMD slots. Because the algorithm iterates through approximately $\ell/2$ offsets and each full vector operation requires $\lceil \ell/N \rceil$ batches to process, the total computational complexity of our scheme is $O(\ell^2/N)$. Thus, the average complexity is $O((n_A n_B)^2/n^2 N)$ for FHE-SpGEMM and $O((n_A n_B n_C)^2/n^2 N)$ for FHE-SpGEMCM.

4.3 Experimental Evaluation

Experimental Setup To validate our theoretical analysis and compare our scheme with other related state-of-the-art schemes, all evaluations are performed on a high-performance server equipped with dual AMD EPYC 9254 24-Core processors, totaling 48 physical cores and 96 logical threads. Our software implementation utilizes the CKKS scheme within the `Pyfhel` [19] FHE framework, which is based on the Ring Learning With Errors (RLWE) assumption [28] over the cyclotomic ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$.

In our primary configuration, we set the polynomial degree to $N = 2^{13}$, providing 2^{13} SIMD slots per ciphertext. To accommodate the required multiplicative depth d , we select 9 intermediate prime levels, as a larger set of levels enables a more extensive sequence of homomorphic operations. Furthermore, to examine the performance under extreme conditions, we investigate edge cases using an expanded configuration of $d = 13$ and $N = 2^{14}$.

We compare our approach with general matrix multiplication (MM) [9] and sparse matrix-vector multiplication (SpMV Lodia) [41]. MM is a method whose homomorphic cost scales with logical matrix dimensions. While an HE-optimized sparse matrix-vector multiplication scheme. To standardize the experimental conditions, we execute Lodia s times to compute $C = AB$ column-by-column, reflecting a common practical reduction from SpMV to matrix-matrix multiplication.

Performance Comparison with Baselines Before evaluating performance, we assess the numerical stability of our scheme. Our implementation maintains a relative error of approximately 10^{-3} after rescaling indices (up to 2^{14}) into the Chebyshev interval. This precision balances the CKKS scaling factor with circuit depth, providing sufficient precision for structural alignment without requiring double-precision homomorphic evaluation. All results were cross-validated against plaintext implementations, confirming structural correctness.

To facilitate a systematic performance comparison, we categorize the test cases into three groups based on their scales, defined as the dimensions of the matrices: **(i) normal-scale matrices**, where the number of rows or columns does not exceed 2^8 ; **(ii) middle-scale matrices**, where the number of rows or columns are 2^9 ; and **(iii) extreme-scale matrices**, where both the row and column dimensions are fixed to 2^{14} .

Normal-scale matrices. Figure 1 reports the performance results under normal-scale sparse settings. As shown in Figure 1a, we assume matrix $A = B \in \mathbb{R}^{2^6 \times 2^6}$ and vary the dimensions $n_A \times n_B$. our scheme consistently outperforms both MM and Lodia when the nonzero interaction scale $n_A \times n_B$ is below 2^{14} . The reduced execution time is primarily attributed to our scheme’s ability to exploit sparsity by limiting homomorphic operations to nonzero interactions only. This behavior aligns with the analysis in Lemma 1, which bounds the number of encrypted elements involved in the multiplication by $n_A \times n_B$. As the scale decreases, our scheme correspondingly reduces the number of homomorphic rotations and multiplications required for data alignment and accumulation. In

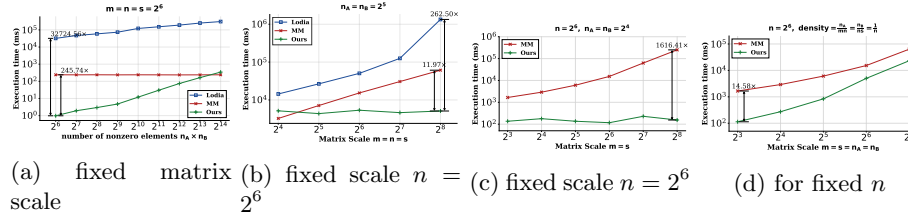


Figure 1: Normal scale performance comparison of our scheme, MM and Lodia: (a-b) Square matrices; (c-d) Non-square matrices.

particular, for $n_A \times n_B = 2^6$, our scheme achieves up to a $245.74\times$ speedup over the baseline methods.

Figure 1b further illustrates the scalability of the three approaches as the matrix dimension $m = n = s$ scales, under a fixed setting of $n_A \times n_B = 2^{10}$ ($n_A = n_B = 2^5$). While the execution times of MM and Lodia grow noticeably with n , the runtime of our scheme remains largely stable across the evaluated range. This behavior arises from the fact that the computational cost of our scheme is not directly determined by the logical matrix dimension, but is instead determined by the nonzero interaction scale ℓ . In contrast, MM exhibits approximately linear growth with respect to n , achieving a speedup of up to $11.97\times$ and $262.50\times$ at $n = 2^8$ compared to MM and Lodia, respectively. By skipping zero-value slots and avoiding unnecessary homomorphic operations, our scheme effectively translates sparsity into a reduction in computational overhead.

To demonstrate the versatility of our approach in non-square configurations, Figure 1c and Figure 1d present the performance results under normal-scale sparse settings. As illustrated in Figure 1c, we fix the number of nonzero entries at $n_A = n_B = 2^4$, $n = 2^6$, to examine the execution time as the matrix dimension $m = s$ scales from 2^3 to 2^8 . While the baseline MM method exhibits a linear increase in execution time with the matrix scale, our scheme maintains remarkably stable performance and lower computational overhead, achieving a speedup of $2788.11\times$. It demonstrates its efficiency even when the matrix dimensions grow rapidly due to its sparse structure. This scalability is further explored in Figure 1d under a constant density setting where $n = 2^6$ and $\frac{n_A}{mn} = \frac{n_B}{ns} = \frac{1}{4}$. Although both methods show an upward trend as the problem size grows, our scheme consistently outperforms the baseline by approximately one order of magnitude. The findings are consistent with expectations: as the matrix parameters m and s double, the value of $n_A \times n_B$ grows by four times, justifying the validity of the results.

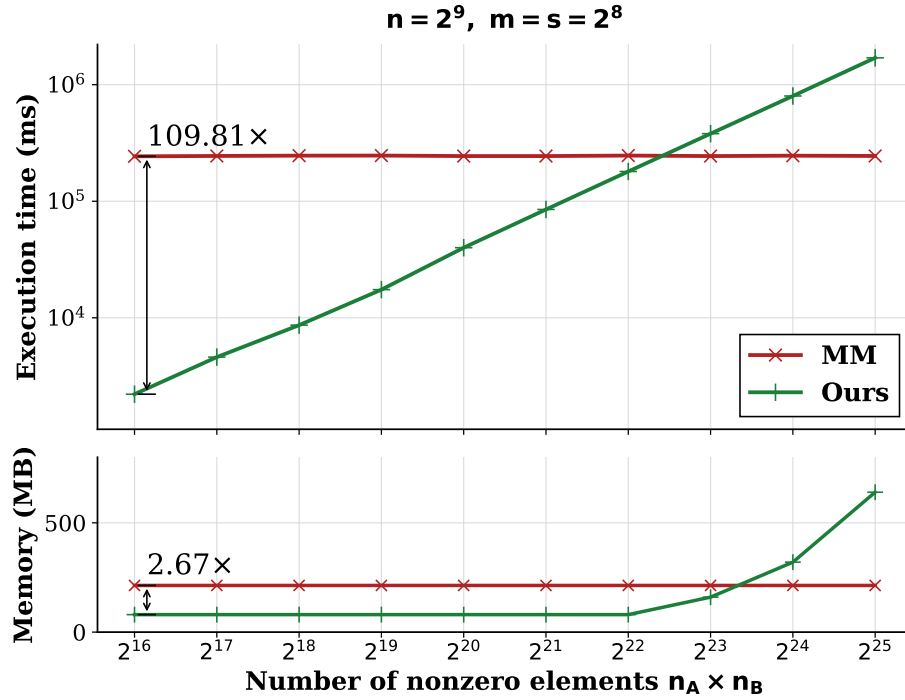
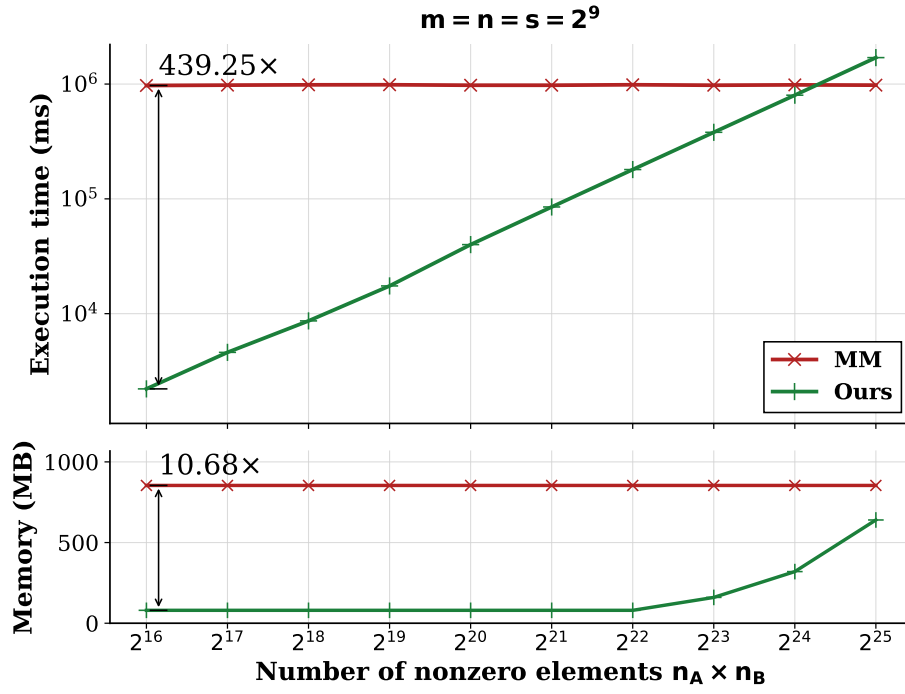
Middle-scale matrices. To evaluate performance under middle-scale matrix dimensions, Figure 2 compares our scheme and the MM baseline under two configurations: (a) square matrices with $m = n = s = 2^9$, and (b) nonsquare matrices with $n = 2^9$ and $m = s = 2^8$. The horizontal axis represents the number of nonzero interactions $n_A \times n_B$, while both execution time and memory consumption are reported.

In the square setting (Figure 2a), MM exhibits a nearly constant execution time of around 10^6 ms, as its homomorphic cost depends on matrix dimensions rather than sparsity. In contrast, our scheme scales with $n_A n_B$, reflecting its dependence on the effective interaction scale. Under extremely sparse settings, our scheme achieves up to $439.25\times$ speedup and remains competitive until the matrix density reaches approximately 12.5%. The memory consumption results show a similar pattern. MM consistently consumes approximately 854 MB regardless of sparsity, whereas our scheme maintains significantly lower memory usage, achieving up to $10.68\times$ reduction. Memory consumption increases only when $n_A n_B$ approaches the dense limit, consistent with the quadratic growth of candidate interactions. The non-square setting (Figure 2b) exhibits the same trend. MM remains insensitive to sparsity, while our scheme scales smoothly with $n_A n_B$, achieving up to $109.81\times$ runtime speedup and $2.67\times$ memory reduction in sparse settings.

Extreme-scale matrices. To evaluate the performance with matrices with extreme dimensions, Figure 3 compares our scheme and Lodia with $A \in \mathbb{R}^{2^{13} \times 2^{13}}$, $B \in \mathbb{R}^{2^{13} \times 1}$, $N = 2^{14}$, and $d = 13$. The MM method is excluded from this experiment, as prior baseline analysis shows that it requires ciphertext partitioning once the matrix dimensions exceed the slot limit, making it unsuitable for direct comparison at this scale.

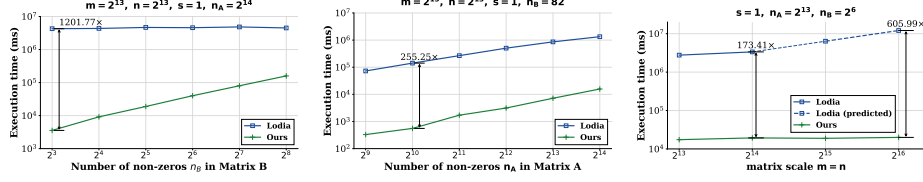
The results demonstrate that our scheme significantly outperforms Lodia across all cases, maintaining execution times 1 to 2 orders of magnitude lower than the baseline. A notable observation in Figure 3a is that the execution time of Lodia remains nearly constant as n_B increases. This indicates that Lodia fails to account for the sparsity of Matrix B, treating the operation as a fixed computational task and failing to benefit from B’s low density. Furthermore, the results reveal that Lodia experiences a dramatic escalation in execution time (i.e., exceeding 10^6 ms in extreme-scale cases) primarily due to its substantial memory demands. In these settings, Lodia’s memory footprint surpasses the 250 GB of physical RAM available on the experimental server, triggering frequent disk I/O operations. This excessive swapping leads to severe latency amplification, ultimately causing the observed performance collapse.

In contrast, our scheme exhibits high sensitivity to the sparsity of both matrices. In Figure 3a, our scheme follows a clear linear growth trend as n_B scales, reaching at most $255.25\times$ speedup. In Figure 3b, with $n_B = 82$ fixed, the speedup further reaches $1201.77\times$. In Figure 3c, when $\ell = n_A \times n_B$ is fixed, our scheme maintains a near-constant execution time even as the logical dimensions m and n scale from 2^{13} to 2^{16} . This consistency aligns with our findings in smaller-scale experiments, indicating that our scheme effectively bounds homomorphic complexity by the nonzero interaction scale ℓ , rather than the logical matrix dimensions, enabling efficient sparse matrix multiplication even at extreme scales.



(b) fixed nonsquare matrix scale $n = 2^9, m = s = 2^8$

Figure 2: Middle scale performance and memory consumption comparison of our scheme and MM



(a) fixed matrix size and n_A (b) fixed matrix size and n_B (c) fixed s and $n_A * n_B$
Figure 3: Extreme scale performance comparison of our scheme and Lodia

5 Discussion on Further Optimizations

The associative property of matrix multiplication provides critical optimization opportunities for our scheme. By selecting the grouping order and execution structure, we can enhance the performance through two primary dimensions: data volume control and noise management.

5.1 Minimizing Intermediate Data Expansion

According to Theorem 1, the number of effective elements ℓ in an encrypted multiplication result is determined by the sparsity patterns of the input matrices, defined as $\ell = \sum_{k=1}^n k_A k_B$. When executing a chain of multiplications, the optimal execution plan must prioritize merging matrix pairs that produce the smallest ℓ to prevent an explosion of encrypted data.

Consider the multiplication of three matrices: $A \in \mathbb{R}^{10^5 \times 10^5}$ (all its nonzero elements are in the first column), $B \in \mathbb{R}^{10^5 \times 10^5}$ (all its nonzero elements are in the first row), and $C \in \mathbb{R}^{10^5 \times 10}$ (highly sparse). There are two distinct computational strategies:

- **Strategy 1:** $(A \times B) \times C$: If $A \times B$ is computed first, the relatively high density of B causes each column of A to intersect with multiple row vectors of B . This results in a dense intermediate matrix AB with a high nonzero count (ℓ_{AB}), which subsequently introduces a massive number of homomorphic rotations and additions when multiplied by C .
- **Strategy 2:** $A \times (B \times C)$: In this execution order, the multiplication between B and C is performed first. Since C is highly sparse and has a relatively small number of rows, the intermediate result BC preserves a sparse structure with a limited number of effective elements. Specifically, only the nonzero elements in the first row of B participate in the multiplication, and their interaction with the sparse rows of C significantly constrains the expansion of nonzero entries. As a result, the effective length ℓ_{BC} remains small, leading to a compact encrypted representation of the intermediate matrix. Subsequently, multiplying A with BC becomes more efficient, as A contains nonzero elements only in its first column. This structure aligns well with the sparsity pattern of BC , minimizing the number of homomorphic multiplications, rotations, and additions required.

More generally, this example highlights that, in FHE-based sparse matrix multiplication, the execution order of chained matrix products can have a significant impact on performance. By carefully reordering matrix multiplications according to sparsity patterns, users can effectively control the expansion of intermediate encrypted data and reduce the number of expensive homomorphic operations. Therefore, selecting an appropriate multiplication order serves as a practical and powerful optimization strategy for reducing the computational overhead in FHE-SpGEMM.

5.2 Noise Management

Beyond data patterns, managing the inherent noise growth in FHE is also critical for performance. Homomorphic multiplication introduces substantial noise, which accumulates rapidly as the multiplicative depth of the circuit increases. For a chain of four matrices $A \times B \times C \times D$, leveraging associativity allows for a choice of different structures:

- **Linear Chain:** A structure such as $((A \times B) \times C) \times D$ results in a multiplicative depth of 3.
- **Balanced Tree:** Reorganizing the operations into $(A \times B) \times (C \times D)$ reduces the multiplicative depth to 2.

Minimizing this depth is of significant practical value, as it reduces the frequency of computationally expensive and memory-intensive bootstrapping operations. Alternatively, it allows for the selection of smaller security parameters, thereby enhancing overall system throughput.

6 Related Works

Sparse matrix–matrix multiplication has been widely optimized in plaintext settings through classical algorithms [31, 36, 42] and specialized hardware accelerators [24, 29, 37], all of which rely on explicit sparsity patterns and data-dependent access—capabilities incompatible with fully homomorphic encryption (FHE). Within encrypted computation, numerous works address matrix multiplication under FHE, but primarily in dense regimes [2, 5, 40]. Gao et al. propose HEGMM and HEGMM-Enhanced [9], which leverage SIMD packing and optimized linear transformations to significantly reduce the cost of general matrix multiplication on encrypted data. However, these methods assume dense matrices and cannot exploit joint sparsity without revealing structural information. For sparse workloads, Yu et al. introduce Lodia [41], the first FHE-compatible sparse matrix–vector multiplication framework. Lodia decomposes a sparse matrix into low-diagonal factors that can be processed efficiently by a diagonal-order encoder, achieving optimal asymptotic complexity while hiding sparsity structure. Yet Lodia focuses on SpMV and does not extend to general sparse matrix–matrix multiplication or chained sparse operations. our scheme differs from these lines of work by enabling privacy-preserving sparse matrix–matrix multiplication that

jointly exploits sparsity in both operands while keeping all structural information hidden. Unlike dense HEGMM-style multiplication and vector-oriented Lodia, our scheme introduces encrypted pre-alignment, SIMD-friendly equality testing, and homomorphic accumulation tailored for SpGEMM, and extends naturally to multi-matrix chains. To our knowledge, it is the first FHE framework to provide fully oblivious, sparsity-aware SpGEMM for general sparse matrices.

7 Conclusion

This paper introduced our scheme, a privacy-preserving sparse matrix multiplication framework that efficiently exploits joint sparsity under FHE. By combining pre-alignment of nonzero elements, SIMD-optimized vector extension, efficient equality checking, and diagonal-aware encrypted accumulation, our scheme enables sparsity aware computation without revealing data values or structural patterns. Our theoretical analysis shows that our scheme achieves a complexity theoretical worst-case upper bound of $O((n_A n_B)^2/N)$ and expected cost $O((n_A n_B)^2/n^2 N)$. Moreover, our method extends naturally to chained multiplications. Experiments demonstrate that our scheme achieves at most $1201.77\times$ speedup while preserving the flexibility of supporting sparse matrices with different shapes. These results highlight our scheme’s potential as a fundamental building block to enable a wide range of privacy-preserving applications that involve sparse matrix multiplications.

References

1. Acar, A., Aksu, H., Uluagac, A.S., Conti, M.: A survey on homomorphic encryption schemes: Theory and implementation. *ACM Computing Surveys (Csur)* **51**(4), 1–35 (2018)
2. Aikata, A., Roy, S.S.: Secure and efficient outsourced matrix multiplication with homomorphic encryption. In: *International Conference on Cryptology in India*. pp. 51–74. Springer (2024)
3. Azad, A., Buluç, A., Gilbert, J.R.: Parallel triangle counting and enumeration using matrix algebra. 2015 IEEE International Parallel and Distributed Processing Symposium Workshop pp. 804–811 (2015), <https://api.semanticscholar.org/CorpusID:2796736>
4. Azad, A., Pavlopoulos, G.A., Ouzounis, C.A., Kyrpides, N.C., Buluç, A.: Hipmcl: a high-performance parallel implementation of the markov clustering algorithm for large-scale networks. *Nucleic acids research* **46**(6), e33–e33 (2018)
5. Bae, Y., Cheon, J.H., Hanrot, G., Park, J.H., Stehlé, D.: Plaintext-ciphertext matrix multiplication and fhe bootstrapping: fast and fused. In: *Annual International Cryptology Conference*. pp. 387–421. Springer (2024)
6. Buluç, A., Gilbert, J.R.: Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments. *SIAM Journal on Scientific Computing* **34**(4), C170–C191 (2012)
7. Frederiksen, T.K., Lindell, Y., Osheter, V., Pinkas, B.: Fast distributed rsa key generation for semi-honest and malicious adversaries. In: *Annual International Cryptology Conference*. pp. 331–361. Springer (2018)

8. Gao, J., Ji, W., Chang, F., Han, S., Wei, B., Liu, Z., Wang, Y.: A systematic survey of general sparse matrix-matrix multiplication. *ACM Computing Surveys* **55**(12), 1–36 (2023)
9. Gao, Y., Quan, G., Homs, S., Wen, W., Wang, L.: Secure and efficient general matrix multiplication on cloud using homomorphic encryption. *The Journal of Supercomputing* **80**(18), 26394–26434 (Aug 2024). <https://doi.org/10.1007/s11227-024-06428-8>, <http://dx.doi.org/10.1007/s11227-024-06428-8>
10. Geelen, R., Vercauteren, F.: Bootstrapping for bgv and bfv revisited. *Journal of Cryptology* **36**(2), 12 (2023)
11. Gilbert, J.R., Moler, C., Schreiber, R.: Sparse matrices in matlab: Design and implementation. *SIAM journal on matrix analysis and applications* **13**(1), 333–356 (1992)
12. Gleyzer, V., Soszynski, A.J., Kao, E.K.: Leveraging linear algebra to count and enumerate simple subgraphs. In: 2020 IEEE High Performance Extreme Computing Conference (HPEC). pp. 1–8. IEEE (2020)
13. Gottlieb, S., Jung, J.H., Kim, S.: A review of david gottlieb’s work on the resolution of the gibbs phenomenon. *Communications in Computational Physics* **9**(3), 497–519 (2011)
14. Gustavson, F.G.: Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Transactions on Mathematical Software (TOMS)* **4**(3), 250–269 (1978)
15. Hazay, C., Lindell, Y.: A note on the relation between the definitions of security for semi-honest and malicious adversaries. *Cryptology ePrint Archive* (2010)
16. Hoefler, T., Alistarh, D., Ben-Nun, T., Dryden, N., Peste, A.: Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research* **22**(241), 1–124 (2021)
17. Hong, S., Kim, S., Choi, J., Lee, Y., Cheon, J.H.: Efficient sorting of homomorphic encrypted data with k-way sorting network. *IEEE Transactions on Information Forensics and Security* **16**, 4389–4404 (2021)
18. Huang, Y., Katz, J., Evans, D.: Quid-pro-quo-tocols: Strengthening semi-honest protocols with dual execution. In: 2012 IEEE Symposium on Security and Privacy. pp. 272–284. IEEE (2012)
19. Ibarrondo, A., Viand, A.: Pyfhel: Python for homomorphic encryption libraries. In: Proceedings of the 9th on workshop on encrypted computing & applied homomorphic cryptography. pp. 11–16 (2021)
20. Idrissi, N., Zellou, A.: A systematic literature review of sparsity issues in recommender systems. *Social Network Analysis and Mining* **10**(1), 15 (2020)
21. Iliashenko, I., Zucca, V.: Faster homomorphic comparison operations for bgv and bfv. *Proceedings on Privacy Enhancing Technologies* **2021**(3), 246–264 (2021)
22. Li, J., Wang, F., Araki, T., Qiu, J.: Generalized sparse matrix-matrix multiplication for vector engines and graph applications. In: 2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC). pp. 33–42. IEEE (2019)
23. Liu, W., Vinter, B.: An efficient gpu general sparse matrix-matrix multiplication for irregular data. In: 2014 IEEE 28th international parallel and distributed processing symposium. pp. 370–381. IEEE (2014)
24. Liu, X., Smelyanskiy, M., Chow, E., Dubey, P.: Efficient sparse matrix-vector multiplication on x86-based many-core processors. In: Proceedings of the 27th international ACM conference on International conference on supercomputing. pp. 273–282 (2013)

25. Liu, Y., Chen, R., Li, S., Yang, J., Li, S., da Silva, B.: Fpga-based sparse matrix multiplication accelerators: From state-of-the-art to future opportunities. *ACM Transactions on Reconfigurable Technology and Systems* **17**(4), 1–37 (2024)
26. Lu, W.j., Huang, Z., Hong, C., Ma, Y., Qu, H.: Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1057–1073. IEEE (2021)
27. Luo, X., Zhou, M., Li, S., Xia, Y., You, Z., Zhu, Q., Leung, H.: An efficient second-order approach to factorize sparse matrices in recommender systems. *IEEE transactions on industrial informatics* **11**(4), 946–956 (2015)
28. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 1–23. Springer (2010)
29. Matam, K., Indarapu, S.R.K.B., Kothapalli, K.: Sparse matrix-matrix multiplication on modern architectures. In: 2012 19th International Conference on High Performance Computing. pp. 1–10. IEEE (2012)
30. Mazzone, F., Everts, M., Hahn, F., Peter, A.: Efficient ranking, order statistics, and sorting under {CKKS}. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 8541–8558 (2025)
31. Pal, S., Beaumont, J., Park, D.H., Amarnath, A., Feng, S., Chakrabarti, C., Kim, H.S., Blaauw, D., Mudge, T., Dreslinski, R.: Outerspace: An outer product based sparse matrix multiplication accelerator. In: 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA). pp. 724–736. IEEE (2018)
32. Paterson, M.S., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM Journal on Computing* **2**(1), 60–66 (1973)
33. Qin, E., Samajdar, A., Kwon, H., Nadella, V., Srinivasan, S., Das, D., Kaul, B., Krishna, T.: Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In: 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). pp. 58–70. IEEE (2020)
34. Rovida, L., Leporati, A., Basile, S.: Lightweight sorting in approximate homomorphic encryption. *Cryptology ePrint Archive*, Paper 2025/1150 (2025), <https://eprint.iacr.org/2025/1150>
35. Trefethen, L.N.: Approximation theory and approximation practice, extended edition. SIAM (2019)
36. Wang, S., Liu, J., Shroff, N.: Coded sparse matrix multiplication. In: International Conference on Machine Learning. pp. 5152–5160. PMLR (2018)
37. Wolfson-Pou, J., Laukemann, J., Petrini, F.: Magnus: Generating data locality to accelerate sparse matrix-matrix multiplication on cpus. In: Proceedings of the 39th ACM International Conference on Supercomputing. pp. 442–457 (2025)
38. Wu, D., Luo, X., Shang, M., He, Y., Wang, G., Zhou, M.: A deep latent factor model for high-dimensional and sparse matrices in recommender systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **51**(7), 4285–4296 (2019)
39. Wu, L., Wang, X.A., Liu, J., Su, Y., Tu, Z., Liu, W., Lei, H., Tang, D., Cao, Y., Zhang, J.: Homomorphic encryption for machine learning applications with ckks algorithms: A survey of developments and applications. *Computers, Materials, & Continua* **85**(1), 89 (2025)
40. Yang, Y., Li, Z., Ding, Y., Hu, M.: Mat-fhe: arbitrary dimension matrix multiplication scheme for floating point over fully homomorphic encryption. *Cybersecurity* **8**(1), 48 (2025)
41. Yu, J., Chen, K., Fan, X., Chen, Y., Zhu, X., Chen, W.: Lodia: Towards optimal sparse matrix-vector multiplication for batched fully homomorphic encryption. In:

- Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 3649–3663 (2025)
42. Yuster, R., Zwick, U.: Fast sparse matrix multiplication. *ACM Transactions On Algorithms (TALG)* **1**(1), 2–13 (2005)